IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Regularizar: frenar al modelo

Teoría, práctica y, al final, material de consulta

Sprint 7 · S07_02 · teoría breve, 35 min de práctica y material de consulta

EL PROBLEMA

Muchas columnas, pocas filas

120

reservas para entrenar: el alojamiento es nuevo

35

columnas, tras pasar las categorías a 0/1

665

columnas al añadir todas las combinaciones de grado 2

Así crecen las columnas: con grado 2, de **a, b** salen **a, b, a^2, a·b, b^2**: de 2 columnas a 5.

Más columnas que filas: el modelo puede memorizar el ruido, como quien se aprende el examen de memoria.

SOBREAJUSTE

Train perfecto, test mediocre

x	precio real	recta	polinomio grado 4
1	62	62.6	62.0
2	70	67.3	70.0
3	68	72.0	68.0
4	79	76.7	79.0
5	81	81.4	81.0
6 (nueva)	—	86.1	7.0

5 puntos, 5 coeficientes: el polinomio acierta los cinco… y para x = 6 predice 7.

El detector es la **brecha**: R^2 en train menos R^2 en test. Cerca de 0 es sano; grande, el modelo **memoriza**.

LA IDEA

Castigar los coeficientes grandes

30 reservas simuladas con 4 variables **escaladas**: solo **suite** y **alta** influyen; las otras dos son ruido.

Coeficientes	suite	alta	ruido 1	ruido 2
Sin penalizar	5.23	3.11	0.23	0.67
Ridge, alpha 1	5.05	3.00	0.21	0.60
Ridge, alpha 10	3.87	2.29	0.08	0.22
Lasso, alpha 0.5	4.61	2.51	0	0
Lasso, alpha 2	3.14	1.03	0	0

Ridge (L2)

Castigo: **alpha** · suma de **cuadrados**. Encoge todos los coeficientes; ninguno llega a 0.

Lasso (L1)

Castigo: **alpha** · suma de **valores absolutos**. Lleva a 0 los que no sirven.

ELEGIR ALPHA

Elegir alpha sin mirar el test

alpha pequeño

Casi no castiga: se parece a la regresión normal y **sobreajusta**.

alpha grande

Castiga mucho: coeficientes casi 0 y un modelo que se queda **corto**.

El punto medio

Lo busca la **validación cruzada dentro del entrenamiento**.

RidgeCV y **LassoCV** prueban una lista de valores de **alpha** y se quedan con el mejor.

El test no se toca hasta el final: si eliges alpha mirando el test, el examen ya no vale.

RIDGE, LASSO O ELASTICNET

Ninguno gana siempre

Ridge

Mejor cuando **muchas** variables aportan un poco.

Lasso

Mejor cuando **pocas** lo explican casi todo, y además se explica fácil.

ElasticNet

Mezcla las dos penalizaciones: útil con variables muy **correlacionadas**.

Las dos castigan el **tamaño** del coeficiente: por eso las variables se **escalan** antes.

Se compara con validación y se mira el test una sola vez, al final.

PRÁCTICA · CUADERNO S07_02

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación y ejemplo	Un alojamiento nuevo con 120 reservas y un modelo con 665 columnas	Clase
Ejercicio 1	Medir la brecha de sobreajuste del modelo sin regularizar	Clase
Ejemplo	Trayectorias de los coeficientes: Ridge frente a Lasso	Clase
Ejercicio 2	Ridge con **alpha** elegido por validación cruzada (**RidgeCV**)	Clase
Ejercicio 3	Lasso: R^2 en test y cuántos coeficientes se quedan (**LassoCV**)	Clase
Ejemplo	Qué variables ha conservado Lasso	Clase
Ejemplo	Tabla comparativa de los tres modelos	Casa
Mini-reto	Curva de R^2 según el número de reservas	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**: ejercicios 1 a 3, comparativa y mini-reto. Cada uno trae su código y sus aclaraciones.

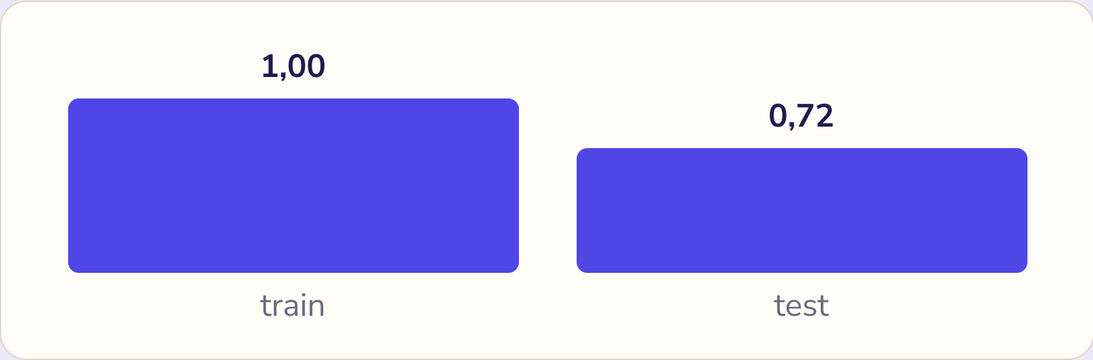
Ejercicio	Tema	Diapositivas
1	Brecha de sobreajuste	9 y 10
2	Ridge y alpha	11 y 12
3	Lasso	13 y 14
Comparativa (casa)	Los tres modelos	15 y 16
Mini-reto (casa)	¿Cuántos datos hacen falta?	17 y 18

CONSULTA · EJERCICIO 1

Medir la brecha de sobreajuste (ejercicio 1)

```
r2_train_ols = ols.score(X_train, y_train)
r2_test_ols = ols.score(X_test, y_test)
brecha_ols = r2_train_ols - r2_test_ols
```

R^2 de **ols** (sin regularizar):



Brecha

1,000 – 0,716 = **0,284**

Un modelo sano rinde parecido en train y en test.

Remedios: **penalizar** (Ridge, Lasso) o **más datos**.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la brecha (ejercicio 1)

score de un modelo

modelo.score(X, y) da el **R²**: con **X_train** mide lo que memorizó; con **X_test**, lo que generaliza.

La brecha

R² train – R² test: cuánto peor va con datos nuevos. Lo sano es **cerca de 0**; 0,28 es sobreajuste.

R² train = 1,000

Un R² de 1 con 665 columnas y 120 filas **no es buena noticia**: el modelo se sabe las respuestas.

Dos remedios

Penalizar los coeficientes (Ridge y Lasso) o **conseguir más datos** (el mini-reto).

CONSULTA · EJERCICIO 2

Alpha por validación cruzada (ejercicio 2)

```
ridge = modelo_con(RidgeCV(alphas=np.logspace(-1, 4, 30))).fit(X_train, y_train)
alpha_ridge = ridge[-1].alpha_
r2_test_ridge = ridge.score(X_test, y_test)
```

np.logspace(-1, 4, 30) = 30 valores entre 0,1 y 10 000 que crecen por factores (como **np.logspace(0, 3, 4)**: 1, 10, 100, 1000).

25,9
alpha elegido por **RidgeCV**

0,728
R² en test con Ridge

0,716
R² en test sin regularizar

El test no se toca hasta el final: si eliges alpha mirando el test, el examen ya no vale.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer RidgeCV (ejercicio 2)

np.logspace(-1, 4, 30)

30 valores entre 0,1 y 10 000 que crecen por **factores**: así se prueban alphas muy distintos.

ridge[-1]

El pipeline tiene varios pasos; **[-1]** es el último, el Ridge, y su **.alpha_** es el **valor elegido**.

Solo un poco mejor

De 0,716 a 0,728: Ridge **encoge** las 665 variables pero no quita ninguna, así que no resuelve el sobreajuste.

Dentro de train

RidgeCV valida **dentro del entrenamiento**: el test queda sin tocar y su R^2 sigue siendo honesto.

CONSULTA · EJERCICIO 3

Lasso: selección de variables (ejercicio 3)

```
lasso = modelo_con(LassoCV(cv=5, max_iter=50000, random_state=SEMILLA)).fit(X_train, y_train)
r2_test_lasso = lasso.score(X_test, y_test)
n_no_cero = int(np.sum(lasso[-1].coef_ != 0))
```

Las 3 que más pesan (cuaderno)	coef.
cat__tipo_habitacion_suite	16.16
cat__tipo_habitacion_estandar	−14.03
num__temporada_alta	7.18

Resultado en test

R^2 con Lasso: **0,887**

Coeficientes distintos de 0: **27 de 665**

Las más importantes son el tipo de habitación y la temporada: justo lo que mueve el precio.

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer LassoCV (ejercicio 3)

LassoCV(cv=5, ...)

Elige **alpha** con validación cruzada de 5 grupos. **max_iter** y **random_state** evitan avisos y fijan el resultado.

Contar los no ceros

lasso[-1].coef_ != 0 marca los que no son 0; **np.sum** los cuenta: 27 de 665, **≈ 4 %**.

Por qué gana a Ridge

Pocas variables lo explican casi todo: Lasso **descarta 638 columnas de ruido**; Ridge las conserva todas, encogidas.

Modelo disperso

Con 27 coeficientes es un modelo mucho **más simple y fácil de explicar**.

CONSULTA · COMPARATIVA

Comparar los tres modelos (casa)

Modelo	R^2 train	R^2 test
Sin regularizar	1.000	0.716
Ridge	0.998	0.728
Lasso	0.957	0.887

Fíjate en la brecha: Lasso baja un poco en train (0,957) y gana mucho en test.

Resultados ilustrativos: en otros datos podría ganar Ridge.

CONSULTA · ACLARACIONES · COMPARATIVA

Leer la comparativa (casa)

La brecha de cada uno

Sin regularizar: 1,000 → 0,716. Ridge: 0,998 → 0,728. Lasso: 0,957 → 0,887.

Lasso pierde en train

Ya **no memoriza**: baja un poco en entrenamiento y gana mucho en prueba, que es lo que importa.

Escalar antes de penalizar

Ridge y Lasso castigan el **tamaño** del coeficiente: sin escalar, castigarían según la unidad de cada variable.

Ilustrativo

En otros datos podría ganar Ridge: se elige **por validación**, no por costumbre.

CONSULTA · MINI-RETO

¿Cuántos datos hacen falta? (mini-reto, casa)

```
tamanos = [60, 120, 300, 1000]
X_res, X_eval, y_res, y_eval = train_test_split(
    X, y, test_size=1000, random_state=SEMILLA)
for n in tamanos:
    # ajusta OLS y Lasso con X_res.iloc[:n], y_res.iloc[:n]
    # y puntúa SIEMPRE con X_eval, y_eval
```

Qué entregas

La tabla **curva**: una fila por tamaño y las columnas **ols** y **lasso**.

Mismo examen para todos

Las mismas **1000 reservas** de evaluación: así la comparación es justa.

Cuidado con n = 300

Con 665 columnas, mínimos cuadrados se vuelve **inestable**: un R^2 muy negativo no es un error.

CONSULTA · ACLARACIONES · MINI-RETO

Leer la curva de tamaños (mini-reto, casa)

Mismo examen

`test_size=1000` aparta **1000 reservas fijas** de evaluación; el resto sirve para entrenar.

iloc[:n]

Toma las **primeras n filas**: cada tamaño parte de los mismos datos.

n = 300 inestable

Con 300 filas y 665 columnas, mínimos cuadrados se dispara: **$R^2 = -1{,}06$** no es un error del cuaderno.

Cuántos datos hacen falta

Con 60: Lasso 0,691 frente a 0,449. Con 1000: 0,925 frente a 0,836. **La ventaja se reduce con más datos.**